



CMPT 295

Unit - Machine-Level Programming

Lecture 24 – Floating-point data & operations

Last lecture

- What is a **buffer overflow**
 - **Defn**: When a function writes more data in array on the **stack** than the array can hold
 - **Effect**: Data kept on the **stack** (value of other local variables, saved register values and return address) may be **corrupted**
- > **Stack smashing**
- Why does buffer overflow spell **trouble**?
Because it creates vulnerability by allowing **hacker attacks**
- How to protect system against such attacks
 1. Avoid creating **overflow vulnerabilities** in the code that we write
 - By always checking array bounds and calling “safe” library functions that consider size of array
 2. Employ system-level protections
 - Randomized initial **stack** pointer and non-executable code segments
 3. Use compiler (like `gcc`) security features:
 - **Stack canary** value and **endbr64** instruction

s/w developer

system

compiler

Today's Menu

- Introduction
 - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
 - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
 - Overview of Function Call
 - Memory Layout and Stack - x86-64 instructions and registers
 - Passing control
 - Passing data – Calling Conventions
 - Managing local data
 - Recursion
- Array
- Buffer Overflow
- Floating-point data & operations

Background

- Once upon a time in the '90's ...
 - Use of computer graphics and image processing (multimedia) applications were on the rise
 - Machine instruction sets were designed and added to ISA's of current microprocessors to support such applications
 - **One idea**: speed up microprocessors by executing single **i**nstruction on **m**ultiple **d**ata -> **SIMD**
- Since then, microprocessors and their machine instruction sets have evolved ...
 - **SSE** (**S**treaming **S**IMD **E**xtensions)
 - **AVX** (**A**dvanced **V**ector **E**Xtensions) -> textbook

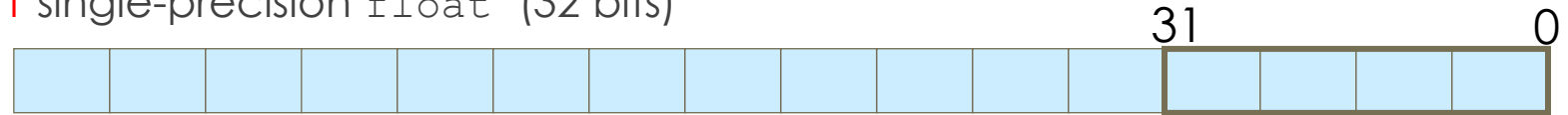
x86-64 ISA extension

- So far, we have covered the *integer registers* and the *integer instructions* of x86-64
 - And the idea -> *SISD*
- Now we shall have a very brief look at the x86-64 ISA extension called *SSE*
 - *SSE* (*S*teaming *S*IMD *E*xtensions) is a single instruction, multiple data (*SIMD*) instruction set extension to the x86 architecture
 - *SSE registers* -> *XMM* registers
 - *SSE instructions* -> *scalar* and *packed* integral/floating-point instructions

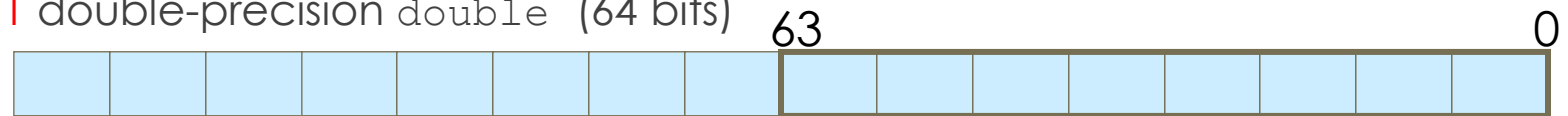
XMM Registers

- 16 in total, each 16-byte wide (128 bits), named: `%xmm0`, `%xmm1`, ..., `%xmm15`

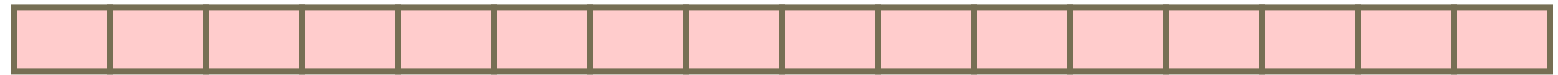
- 1 single-precision `float` (32 bits)



- 1 double-precision `double` (64 bits)



- 16 single-byte integers



- 8 16-bit integers



- 4 32-bit integers



- 4 single-precision `float`'s



- 2 double-precision `double`'s

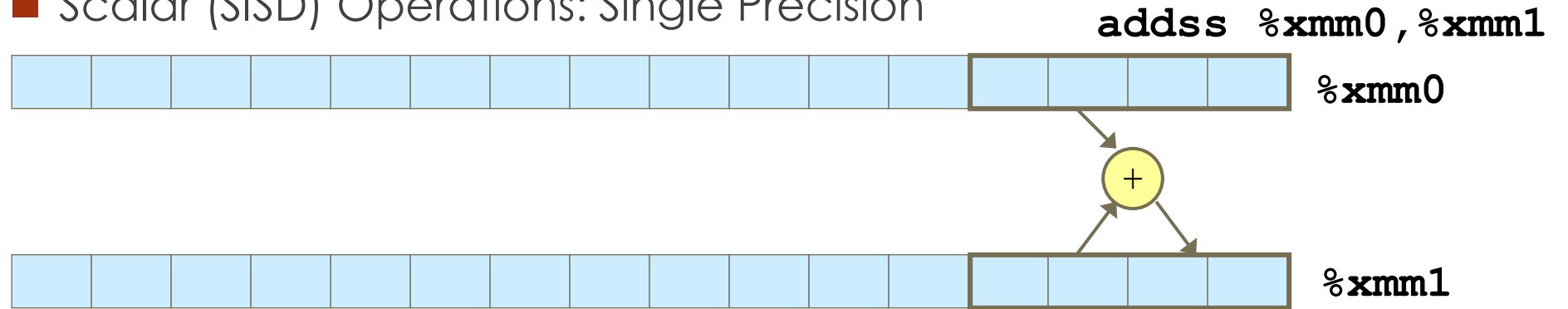


Scalar mode

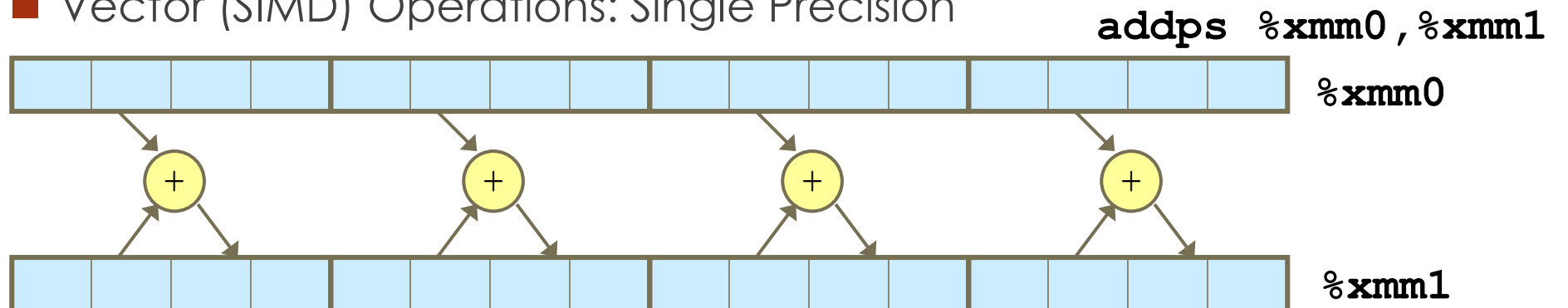
Vector mode
(packed data)

Scalar (SISD) versus Vector (SIMD) instructions

■ Scalar (SISD) Operations: Single Precision



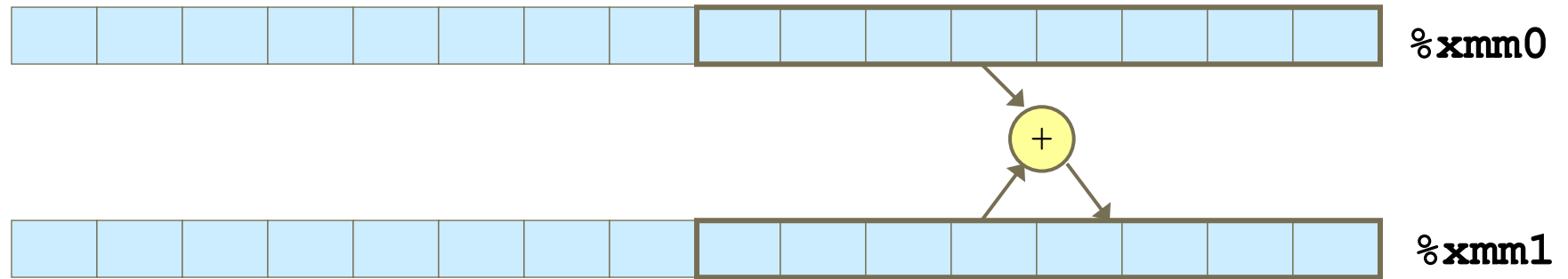
■ Vector (SIMD) Operations: Single Precision



Scalar versus Vector (SIMD) instructions

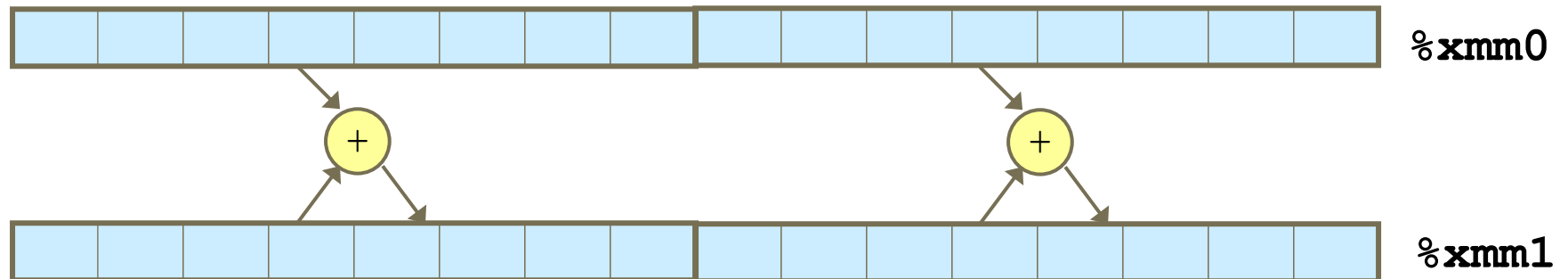
■ Scalar (SISD) Operations: Double Precision

addsd %xmm0, %xmm1



■ Vector (SIMD) Operations: Double Precision

addpd %xmm0, %xmm1



SSE Data movement instructions

```
float_mov:
# -----
# float float_mov(float f1,
#                 float *src,
#                 float *dst) {
#   float f2 = *src;
#   *dst = f1;
#   return f2;
# }
# -----
# f1 in %xmm0, src in %rdi, dst in %rsi
movss  (%rdi), %xmm1  # f2 = *src
movss  %xmm0, (%rsi)  # *dst = f1
movaps %xmm1, %xmm0   # return value = f2
ret
```

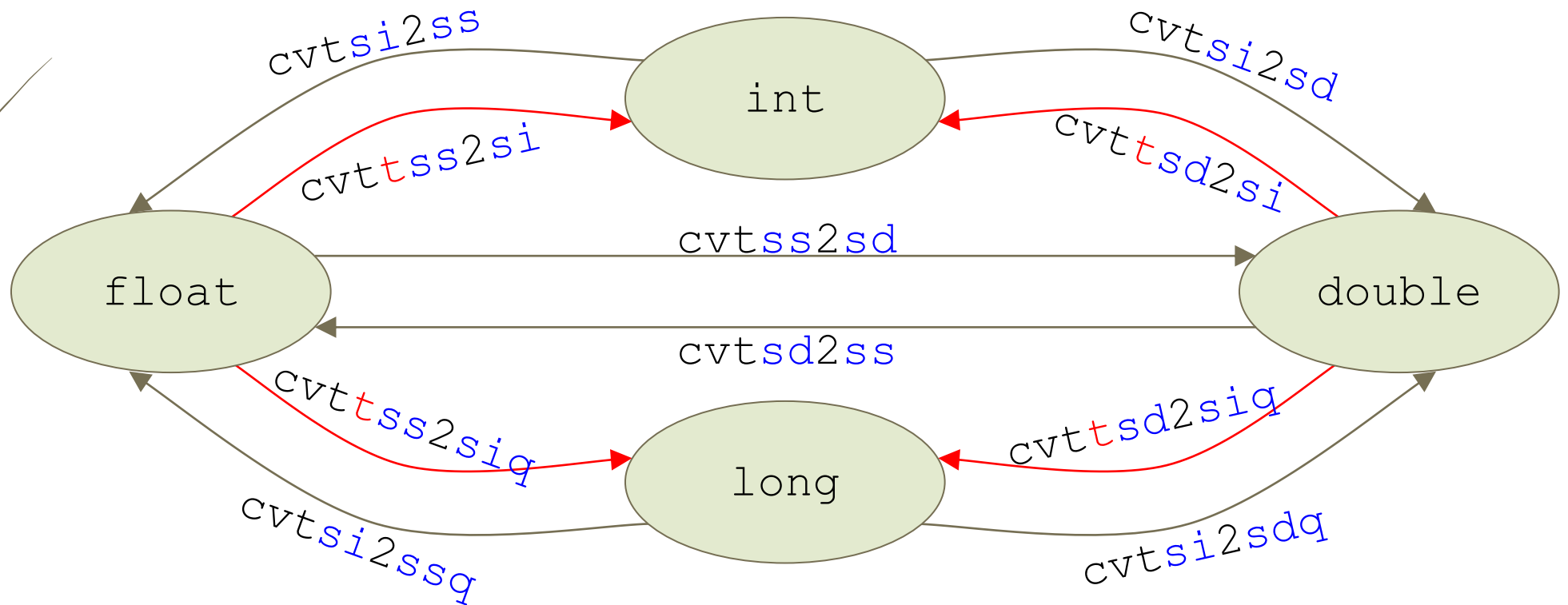
- The instructions we shall look at in this lecture are different than the ones presented in section 3.11 of our textbook – we shall focus on the **scalar** version of these instructions
- **movss** – move single precision
 - Mem (32 bits) <--> %xmm
- **movsd** – move double precision
 - Mem (64 bits) <--> %xmm
- **First 2 instructions of program:** Memory referencing operands (i.e., memory addressing mode operands) specified in the same way as for the integer **mov*** instructions
- **movaps/movapd** – move %xmm <--> %xmm
 - **ap** -> aligned packed

SSE Function call and register saving conventions

- Function call convention
 - Integer (and pointer i.e., memory address) arguments passed in *integer* registers
 - Floating point values passed in XMM registers
 - Argument 1 to argument 8 passed in `%xmm0`, `%xmm1`, ..., `%xmm7`
 - Result returned in `%xmm0`
- Register saving convention
 - All XMM registers **caller-saved**
 - Can use register `%xmm8` ↔ `%xmm15` for managing local data

SSE Data conversion instructions

➡ Converting between data types: (“t” is for “truncate”)



SSE Data manipulation instructions

Arithmetic

- **addss/addsd** - floating point add
- **subss/subsd** - ... subtract
- **mulss/mulsd** - ... mul
- **divss/divsd** - ... div

Logical

- **andps/andpd**
- **orps/d**
- **xorps/d**
 - **xorpd %xmm0, %xmm0**
effect `%xmm0 <- 0`

Comparison: **ucomiss/d**

- Affects only condition codes: CF, ZF
 - use unsigned branches
- If NaN, set all of condition codes: CF, ZF and PF
 - Use `jp/jnp` to branch on PF

Others

- **maxss/maxsd** - ... max
For example: `maxss %xmm3, %xmm5`
Effect: `xmm5 ← max(xmm5, xmm3)`
- **minss/minsd** - ... min
- **sqrtps/sqrtsd** - ... square root

Example

```
fadd:
# -----
# float fadd(float x, float y){
#     return x + y;
# }
# -----
# x in %xmm0, y in %xmm1
    addss    %xmm1, %xmm0
    ret

dadd:
# -----
# double dadd(double x, double y){
#     return x + y;
# }
# -----
# x in %xmm0, y in %xmm1
    addsd    %xmm1, %xmm0
    ret
```

Interesting Example

x86-64 gcc 11.4 -O1

```
int sumArray(int *A, int N) {  
    int sum = 0;  
  
    for( unsigned int i = 0; i < N; i++ )  
        sum += A[i];  
  
    return sum;  
}
```

```
sumArray:  
    testl    %esi, %esi  
    je       .L4  
    movq     %rdi, %rax  
    leal     -1(%rsi), %edx  
    leaq     4(%rdi,%rdx,4), %rcx  
    movl     $0, %edx  
  
.L3:  
    addl     (%rax), %edx  
    addq     $4, %rax  
    cmpq     %rcx, %rax  
    jne      .L3  
  
.L1:  
    movl     %edx, %eax  
    ret  
  
.L4:  
    movl     $0, %edx  
    jmp      .L1
```

Interesting Example

x86-64 gcc 11.4 -O2

```
int sumArray(int *A, int N) {  
    int sum = 0;  
  
    for( unsigned int i = 0; i < N; i++ )  
        sum += A[i];  
  
    return sum;  
}
```

```
sumArray:  
    testl    %esi, %esi  
    je      .L4  
    leal    -1(%rsi), %eax  
    leaq    4(%rdi,%rax,4), %rdx  
    xorl    %eax, %eax  
  
.L3:  
    addl    (%rdi), %eax  
    addq    $4, %rdi  
    cmpq    %rdx, %rdi  
    jne     .L3  
    ret  
  
.L4:  
    xorl    %eax, %eax  
    ret
```

Interesting Example x86-64 gcc 11.4 -O3

sumArray:

```
    movq    %rdi, %rdx
    testl   %esi, %esi
    je      .L7
    leal    -1(%rsi), %eax
    cmpl    $2, %eax
    jbe     .L8
    movl    %esi, %ecx
    movq    %rdi, %rax
    pxor    %xmm0, %xmm0
    shrl    $2, %ecx
    subl    $1, %ecx
    salq    $4, %rcx
    leaq    16(%rdi,%rcx), %rcx
```

.L4:

```
    movdqu  (%rax), %xmm2
    addq    $16, %rax
    padd    %xmm2, %xmm0
    cmpq    %rcx, %rax
    jne     .L4
    movdqa  %xmm0, %xmm1
    movl    %esi, %ecx
    psrldq  $8, %xmm1
    andl    $-4, %ecx
    padd    %xmm1, %xmm0
    movdqa  %xmm0, %xmm1
    psrldq  $4, %xmm1
    padd    %xmm1, %xmm0
    movd    %xmm0, %eax
    testb   $3, %sil
    je      .L11
```

Interesting Example

x86-64 gcc 11.4 -O3 - cont'd

```
.L3:
    movl    %ecx, %edi
    addl    (%rdx,%rdi,4), %eax
    leal    1(%rcx), %edi
    cmpl    %esi, %edi
    jnb     .L1
    addl    $2, %ecx
    addl    (%rdx,%rdi,4), %eax
    cmpl    %ecx, %esi
    jbe     .L1
    addl    (%rdx,%rcx,4), %eax
    ret

.L7:
    xorl    %eax, %eax

.L1:
    ret

.L11:
    ret

.L8:
    xorl    %ecx, %ecx
    xorl    %eax, %eax
    jmp     .L3
```

Summary

- **SSE** (**S**teaming **S**IMD **E**xtensions) -> **SIMD** instruction set extension to **x86** architecture
 - **XMM** registers:
 - **Scalar** and **vector (packed)** mode of integral and floating point data
 - **SSE instructions**
 - **Scalar** and **vector (packed)** integral/floating-point instructions
- gcc has built-in support for auto-vectorization, which can automatically translate scalar code into SIMD code

Next Lecture

- Instruction Set Architecture (ISA)
 - Definition of ISA
- Instruction Set design
 - Design guidelines
 - Example of an instruction set: MIPS
 - Create our own instruction sets
 - ISA evaluation
- Implementation of a microprocessor (CPU) based on an ISA
 - Execution of machine instructions (datapath)
 - Intro to logic design + Combinational logic + Sequential logic circuit
 - Sequential execution of machine instructions
 - Pipelined execution of machine instructions + Hazards